4-bit Quantum Ripple-Carry Adder in Qiskit

Abert Kabore

PhD student in Advanced and Equitable Computing

Module 4 assignment: extending the supplied 1-bit and 2-bit full adders

1. Design and simulation method

The solution retains the class notebook's SUM, CARRY, and reverse-CARRY gate sequences. It extends the 2-bit layout to four bits using 3n + 1 = 13 qubits. The operands a and b are unsigned 4-bit integers (0 to 15), with an optional carry-in of 0 or 1. The five output bits represent a + b + carry-in, including the final carry-out.

Bit i	Carry c_i	Operand a_i	b_i becomes sum s_i
0	q0 (carry-in)	q1	q2
1	q3	q4	q5
2	q6	q7	q8
3	q9	q10	q11
Final carry	q12 = s4		

First, four CARRY blocks propagate the carry upward. At the highest bit, a CNOT restores b3 before SUM writes s3. Then reverse-CARRY and SUM run downward for bits 2, 1, and 0. This restores the internal carry qubits q3, q6, and q9 to zero while preserving a and the initial carry-in. The arithmetic circuit contains 14 Toffoli (CCX) and 16 CNOT (CX) gates.

Sum bit si is measured into classical bit i, and q12 into bit 4. Qiskit displays bit 4 on the left, so labels read s4s3s2s1s0. For example, 1011 + 0110 = 10001 (11 + 6 = 17).

All measured results use the local ideal Qiskit Aer SamplerV2, 1,024 shots per circuit, seed 4, and no noise model. Counts come from get_counts(); expected sums are computed separately for comparison. These are simulator results. The exhaustive check uses unmeasured Qiskit Statevector circuits.

Executed notebook environment

```
import platform
from importlib.metadata import version

print(f"Python {platform.python_version()}")
for package in ("qiskit", "qiskit-aer", "matplotlib", "numpy"):
    print(f"{package} {version(package)}")
```

```
Python 3.11.0
qiskit 2.5.2
qiskit-aer 0.17.2
matplotlib 3.9.2
numpy 2.3.2
```

2. Code: imports and original arithmetic blocks

```
from qiskit import QuantumCircuit
from qiskit_aer.primitives import SamplerV2
from qiskit.visualization import plot_histogram

# Define functions for the ripple carry adder.
def sum(circ, cin, a, b):
    circ.cx(a,b)
    circ.cx(cin,b)

def carry(circ, cin, a, b, cout):
    circ.ccx(a, b, cout)
    circ.cx(a, b)
    circ.ccx(cin, b, cout)

def carry_rev(circ, cin, a, b, cout):
    circ.ccx(cin, b, cout)
    circ.cx(a, b)
    circ.ccx(a, b, cout)
```

General adder: input preparation

```
from qiskit import QuantumRegister, ClassicalRegister
from qiskit.quantum_info import Statevector
from qiskit_aer.primitives import SamplerV2
import matplotlib.pyplot as plt


def adder_qubits(i):
    """Return (c_i, a_i, b_i), using the original 2-bit qubit layout."""
    return 3 * i, 3 * i + 1, 3 * i + 2


def ripple_carry_adder(n, a, b, cin=0, measure=True):
    """Build an n-bit ripple-carry adder computing a + b + cin.

    a, b: integers in [0, 2**n); bit i is loaded into a_i / b_i.
    cin  : initial carry-in (0 or 1), loaded into qubit c_0
    """
    if not (0 <= a < 2**n and 0 <= b < 2**n):
        raise ValueError(f"a and b must be in [0, {2**n - 1}] for a {n}-bit adder")
    if cin not in (0, 1):
        raise ValueError("cin must be 0 or 1")

    qr = QuantumRegister(3 * n + 1, "q")
    qc = QuantumCircuit(qr, name=f"{n}-bit adder")
    cout = 3 * n

    # encode inputs with X gates
    if cin:
        qc.x(0)
    for i in range(n):
        _, ai, bi = adder_qubits(i)
        if (a >> i) & 1:
            qc.x(ai)
        if (b >> i) & 1:
            qc.x(bi)
    qc.barrier()
```

General adder continued: carry, sum, and measurement

```
    # QC carry ... (ripple up)
    for i in range(n):
        ci, ai, bi = adder_qubits(i)
        # c_{i+1} is qubit ci + 3; the last is cout.
        carry(qc, ci, ai, bi, ci + 3)
    qc.barrier()

    # most significant bit: undo the a XOR b from carry, then sum
    ci, ai, bi = adder_qubits(n - 1)
    qc.cx(ai, bi)
    sum(qc, ci, ai, bi)
    qc.barrier()

    # QC carry reverse + sum ... (ripple down)
    for i in range(n - 2, -1, -1):
        ci, ai, bi = adder_qubits(i)
        carry_rev(qc, ci, ai, bi, ci + 3)
        sum(qc, ci, ai, bi)
        qc.barrier()

    if measure:
        cr = ClassicalRegister(n + 1, "s")
        qc.add_register(cr)
        for i in range(n):
            qc.measure(adder_qubits(i)[2], cr[i])   # extract Sum_i
        qc.measure(cout, cr[n])                     # extract Cout (Sum_n)
    return qc


def run_counts(circuits, shots=1024):
    """Run circuits on the Aer sampler and return a list of counts dicts."""
    result = SamplerV2(seed=4).run(circuits, shots=shots).result()
    return [r.data.s.get_counts() for r in result]
```

3. Example: binary 1011 + 0110 = 10001

```
# 4-bit full adder: load binary 1011 and 0110.
a, b = 11, 6
qc = ripple_carry_adder(4, a, b)
print(f"qubits: {qc.num_qubits}, depth: {qc.depth()}")
print(f"gates: {dict(qc.count_ops())}")
qc.draw(fold=120)
```

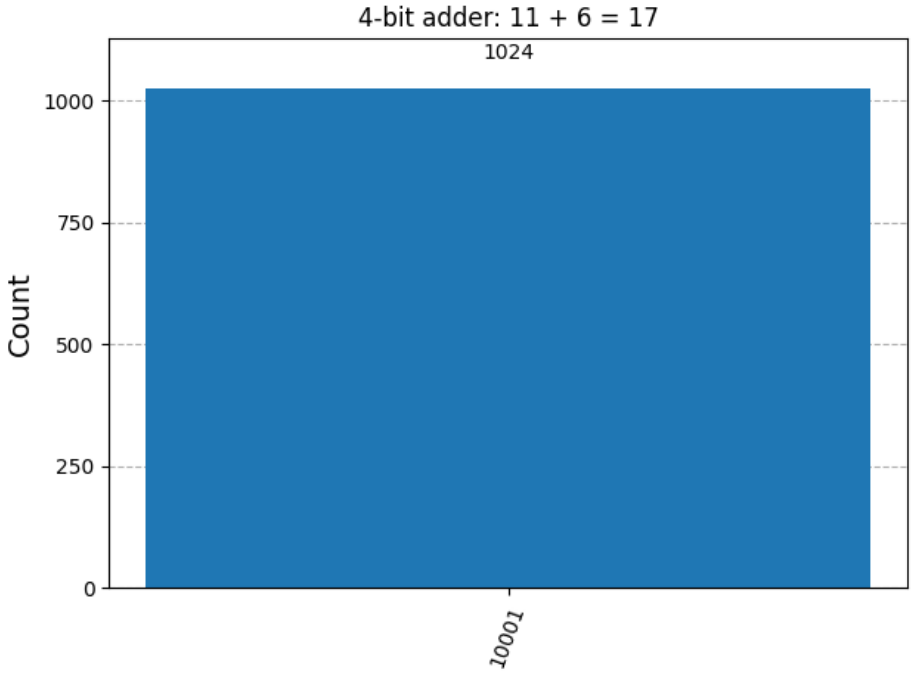
```
qubits: 13, depth: 26
gates: {'cx': 16, 'ccx': 14, 'barrier': 6, 'x': 5, 'measure': 5}
```

```
counts = run_counts([qc])[0]
expected_bits = f"{a + b:05b}"
assert counts == {expected_bits: 1024}, counts
measured = int(max(counts, key=counts.get), 2)
print(f"{a} + {b}: measured {counts} -> {measured} (expected {a + b})")
plot_histogram(counts, title=f"4-bit adder: {a} + {b} = {a + b}")
```

```
11 + 6: measured {'10001': 1024} -> 17 (expected 17)
```

The notebook also displays qc.draw(). The following page shows the histogram saved by the executed example cell.

Example histogram: 11 + 6 = 17



Observations

- All 1,024 shots produced 10001 = 17 for 1011 + 0110 (11 + 6). The carry-out is 1 and the lower four sum bits are 0001.
- Design: The 4-bit solution extends the supplied SUM, CARRY, and reverse-CARRY blocks without changing their gate sequences. It uses 13 qubits, 14 Toffoli (CCX) gates, and 16 CNOT (CX) gates.
- Output interpretation: The inputs are unsigned 4-bit integers from 0 to 15. The five measured bits include the final carry-out; for example, 1011 + 0110 = 10001 (11 + 6 = 17).
- Data provenance: The counts and histogram images were regenerated by executing the displayed Qiskit circuits with the local ideal Aer SamplerV2, seed 4, and no noise model. Expected sums are computed separately for comparison.
- Histogram reading: The vertical axes show shot counts. Each selected test has one bar with 1,024 counts at the expected bitstring, corresponding to relative frequency 1.
- Scope: These results verify unsigned 4-bit addition for basis-state inputs under ideal simulation. Hardware performance and behavior under noise were not evaluated.

4. Selected test cases

a (4 bits)	b (4 bits)	Carry-in	Purpose
0000 (0)	0000 (0)	0	Zero inputs
0101 (5)	0010 (2)	0	No carry generated
0111 (7)	0001 (1)	0	Carry propagation; no overflow
1001 (9)	0111 (7)	0	Overflow into fifth output bit
1111 (15)	1111 (15)	0	Maximum operands
1111 (15)	0000 (0)	1	Carry-in propagates through all bits

On this ideal simulator, basis-state inputs give deterministic outputs. The assertions require the entire counts dictionary to equal the expected five-bit result with 1,024 counts.

Test-case and histogram code

```
test_cases = [  # (a, b, cin)
    (0, 0, 0),
    (5, 2, 0),
    (7, 1, 0),
    (9, 7, 0),
    (15, 15, 0),
    (15, 0, 1),
]

circuits = [ripple_carry_adder(4, a, b, cin) for a, b, cin in test_cases]
all_counts = run_counts(circuits)

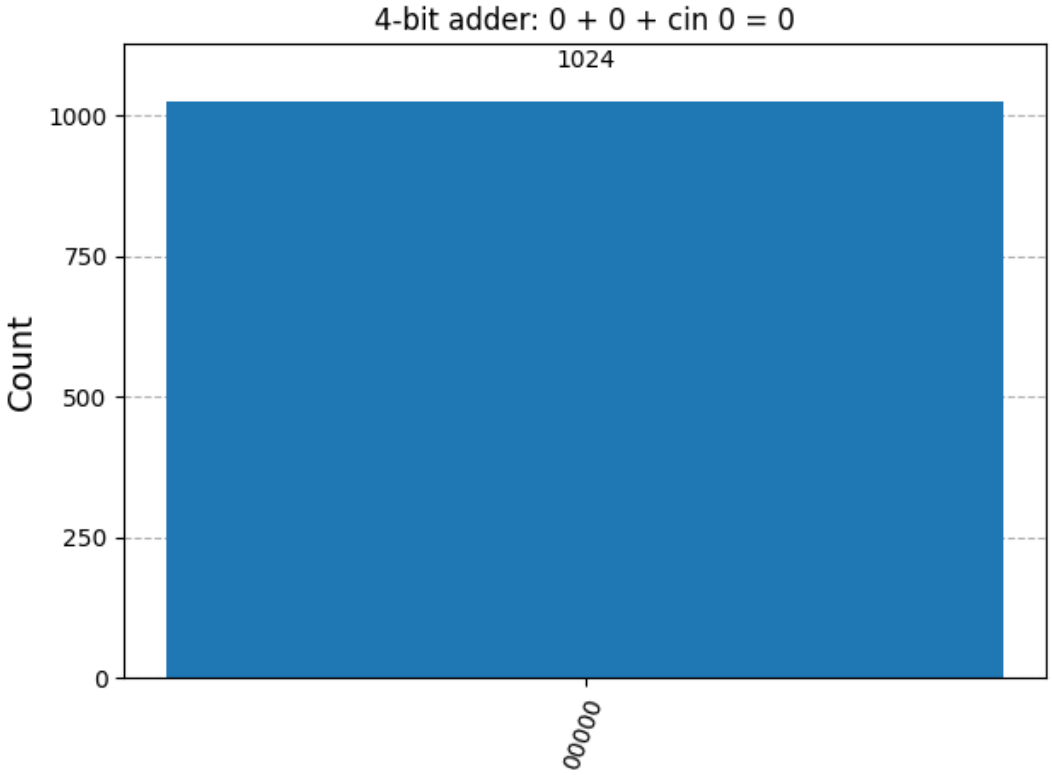
print(f"{'a':>3} {'b':>3} {'cin':>4} | {'expected':>11} | {'measured':>11} | result")
print("-" * 60)
for (a, b, cin), counts in zip(test_cases, all_counts):
    expected = a + b + cin
    measured = int(max(counts, key=counts.get), 2)
    ok = counts == {f"{expected:05b}": 1024}
    assert ok, f"Unexpected simulator counts: {counts}"
    print(
        f"{a:>3} {b:>3} {cin:>4} | {expected:>3} ({expected:05b}) | "
        f"{measured:>3} ({measured:05b}) | {'PASS' if ok else 'FAIL'}"
    )
```

Actual simulator test output

```
  a   b  cin |    expected |    measured | result
-------------------------------------------------------------
  0   0    0 |   0 (00000) |   0 (00000) | PASS
  5   2    0 |   7 (00111) |   7 (00111) | PASS
  7   1    0 |   8 (01000) |   8 (01000) | PASS
  9   7    0 |  16 (10000) |  16 (10000) | PASS
 15  15    0 |  30 (11110) |  30 (11110) | PASS
 15   0    1 |  16 (10000) |  16 (10000) | PASS
```

Test 1: 0 + 0 = 0

```
# Test 1: 0 + 0 = 0
a, b, cin = test_cases[0]
counts = all_counts[0]
title = f"4-bit adder: {a} + {b} + cin {cin} = {a + b + cin}"
plot_histogram(counts, title=title)
```

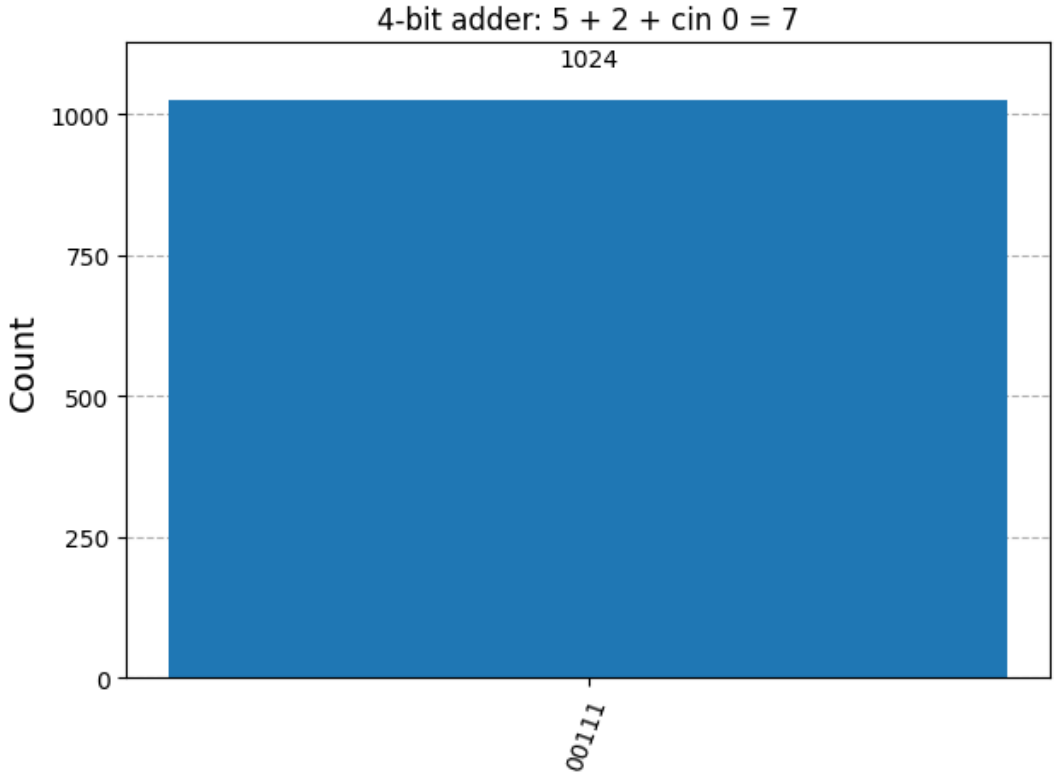


Observations

- All 1,024 shots produced 00000 = 0 for 0000 + 0000 with carry-in 0.
- The sum and carry-out are zero, confirming correct behavior for all-zero inputs.
- Review corrections: Inherited installation logs were replaced with actual environment versions, stored outputs were regenerated, deprecated sampling calls were updated, histogram descriptions were corrected, and a PDF line break inside an f-string was fixed.

Test 2: 5 + 2 = 7

```
# Test 2: 5 + 2 = 7
a, b, cin = test_cases[1]
counts = all_counts[1]
title = f"4-bit adder: {a} + {b} + cin {cin} = {a + b + cin}"
plot_histogram(counts, title=title)
```

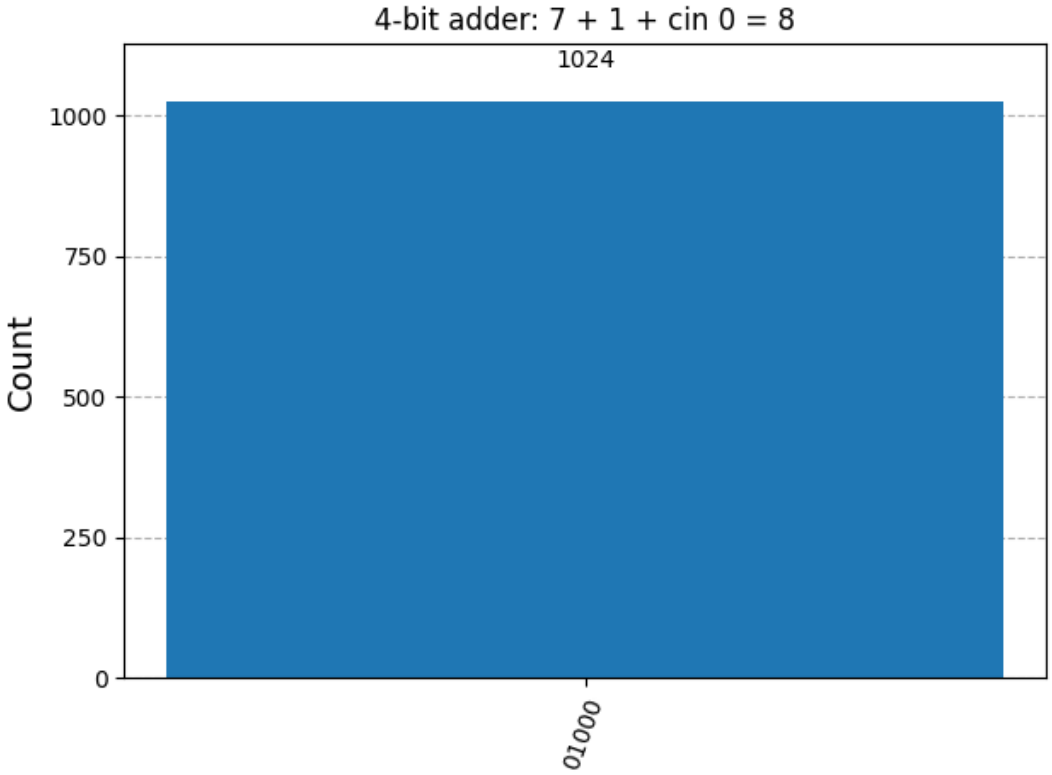


Observations

- All 1,024 shots produced 00111 = 7 for 0101 + 0010 (5 + 2).
- The operands have no overlapping 1 bits, so no carry is generated and the carry-out is 0.

Test 3: 7 + 1 = 8

```
# Test 3: 7 + 1 = 8
a, b, cin = test_cases[2]
counts = all_counts[2]
title = f"4-bit adder: {a} + {b} + cin {cin} = {a + b + cin}"
plot_histogram(counts, title=title)
```

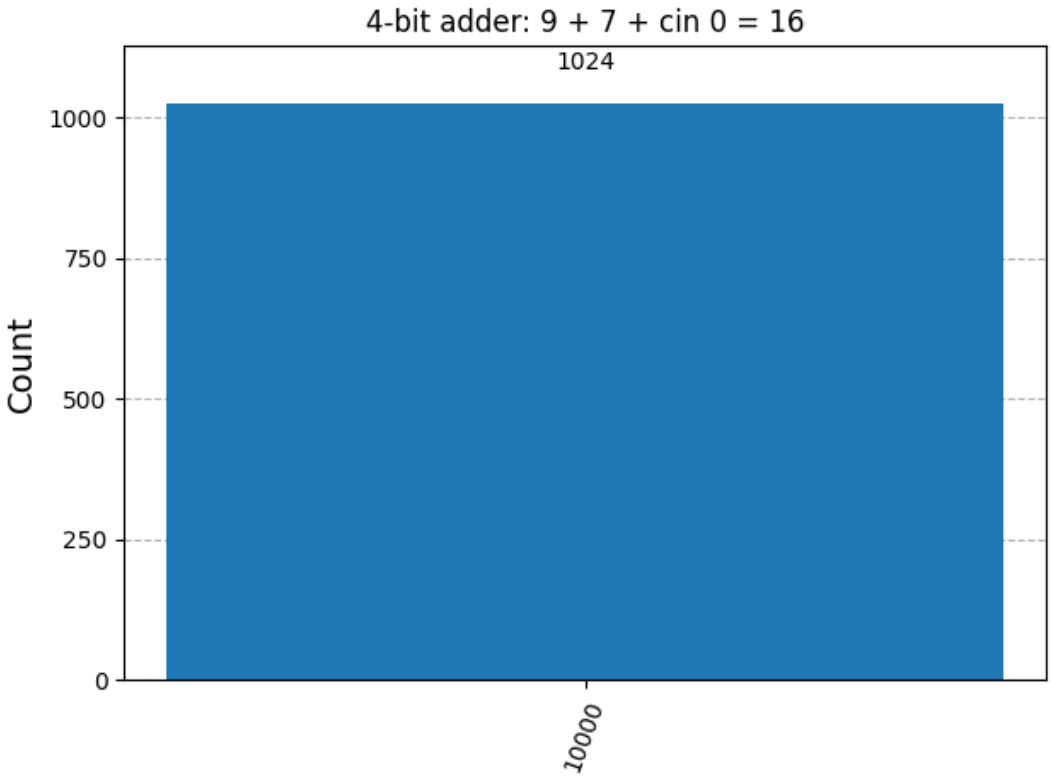


Observations

- All 1,024 shots produced 01000 = 8 for 0111 + 0001 (7 + 1).
- The carry propagates through the lower three bit positions into bit 3; the final carry-out remains 0.

Test 4: 9 + 7 = 16

```
# Test 4: 9 + 7 = 16
a, b, cin = test_cases[3]
counts = all_counts[3]
title = f"4-bit adder: {a} + {b} + cin {cin} = {a + b + cin}"
plot_histogram(counts, title=title)
```

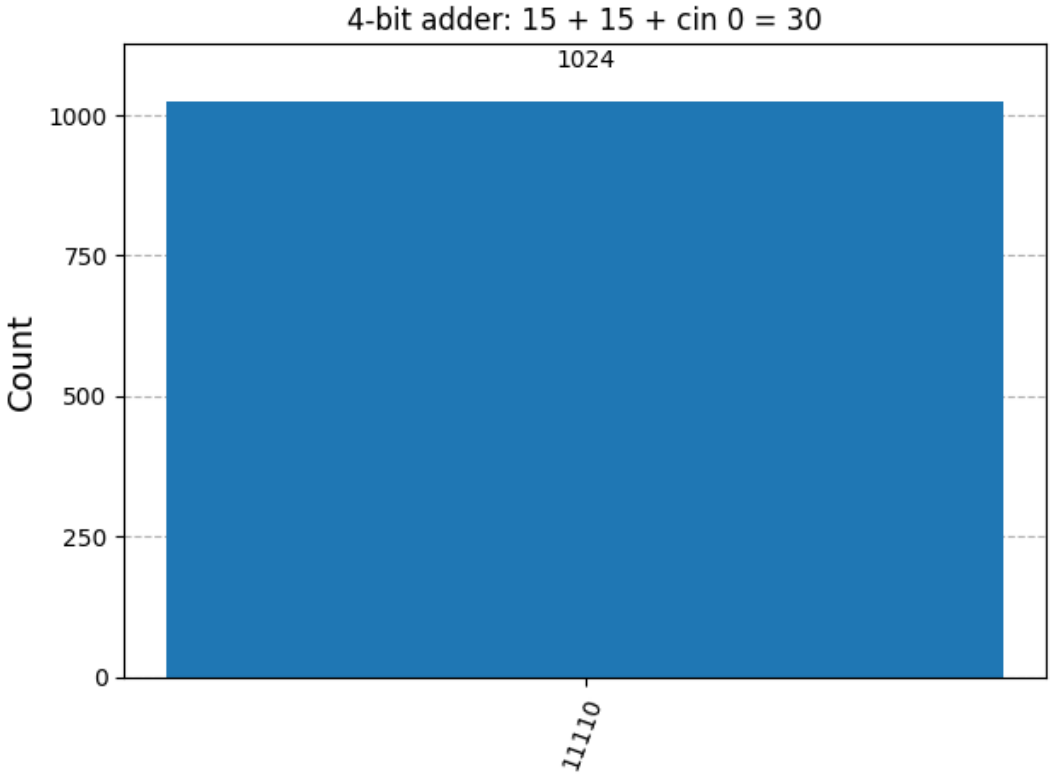


Observations

- All 1,024 shots produced 10000 = 16 for 1001 + 0111 (9 + 7).
- The lower four sum bits are 0000 and the carry-out is 1, confirming overflow into the fifth output bit.

Test 5: 15 + 15 = 30

```
# Test 5: 15 + 15 = 30
a, b, cin = test_cases[4]
counts = all_counts[4]
title = f"4-bit adder: {a} + {b} + cin {cin} = {a + b + cin}"
plot_histogram(counts, title=title)
```

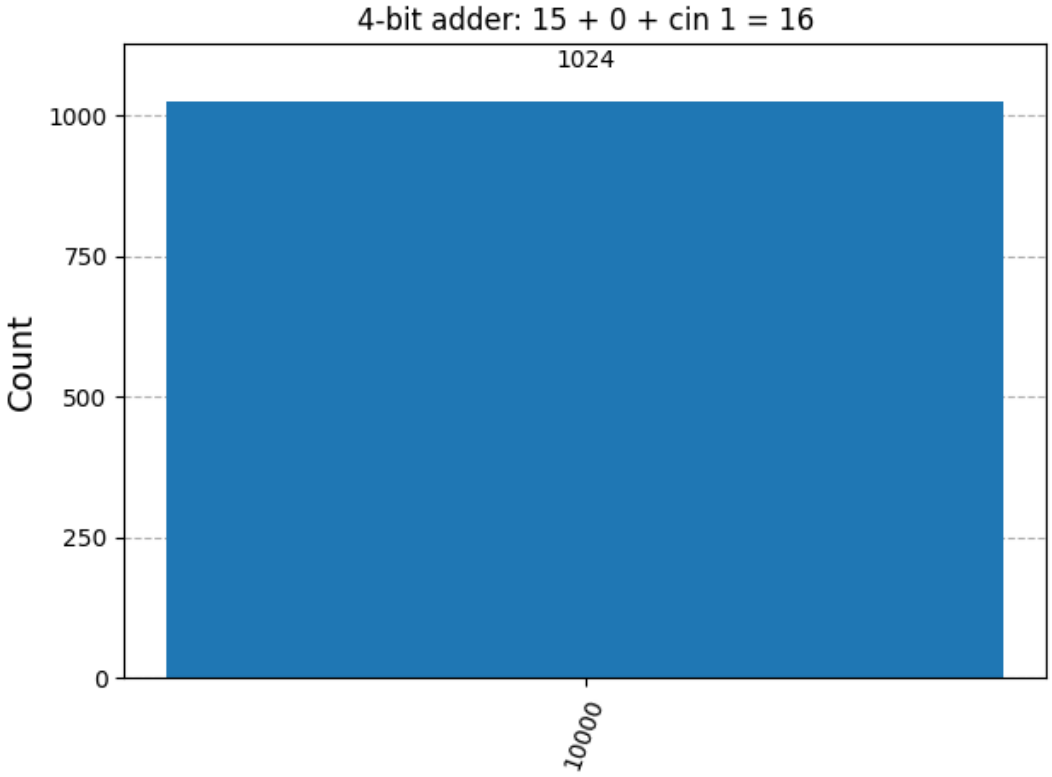


Observations

- All 1,024 shots produced 11110 = 30 for 1111 + 1111 (15 + 15), with carry-in 0.
- Both operands have their maximum 4-bit value. The lower sum bits are 1110 and the carry-out is 1.

Test 6: 15 + 0 + carry-in 1 = 16

```
# Test 6: 15 + 0 + carry-in 1 = 16
a, b, cin = test_cases[5]
counts = all_counts[5]
title = f"4-bit adder: {a} + {b} + cin {cin} = {a + b + cin}"
plot_histogram(counts, title=title)
```



Observations

- All 1,024 shots produced 10000 = 16 for 1111 + 0000 with carry-in 1 (15 + 0 + 1).
- The input carry propagates through all four bit positions, leaving lower sum bits 0000 and carry-out 1.
- Selected tests: All six histogram cases passed with 1,024 shots each. They cover zero inputs, no carry, carry propagation, overflow, maximum operands, and a nonzero carry-in.
- Exhaustive verification: All 512 operand/carry-in combinations passed with zero failures. The check also confirmed that the a register and initial carry-in are preserved and the three internal carry qubits return to zero.
- Assignment coverage: The single PDF includes the complete 4-bit solution code, executed test outputs, and histograms, satisfying the stated submission requirements.

5. Exhaustive verification

The following code checks every 16 x 16 x 2 = 512 combination of a, b, and carry-in. It verifies the five-bit sum, preservation of a and carry-in, and reset of the three internal carry qubits. It also checks that the output is one basis state with probability 1.

```
# sum() above is the quantum SUM block, so use a separate register reader.
def read_register(state, qubits):
    """Read qubits as an integer; the first qubit is least significant."""
    return int("".join(state[q] for q in reversed(qubits)), 2)


n = 4
failures = 0
checked = 0
for cin in (0, 1):
    for a in range(2**n):
        for b in range(2**n):
            circuit = ripple_carry_adder(n, a, b, cin, measure=False)
            probs = Statevector(circuit).probabilities_dict()
            assert len(probs) == 1, "output should be a single basis state"
            assert abs(float(next(iter(probs.values()))) - 1.0) < 1e-12
            state = next(iter(probs))[::-1]  # state[k] is qubit k
            sum_qubits = [adder_qubits(i)[2] for i in range(n)] + [3 * n]
            a_qubits = [adder_qubits(i)[1] for i in range(n)]
            carry_qubits = [adder_qubits(i)[0] for i in range(1, n)]
            s = read_register(state, sum_qubits)
            a_out = read_register(state, a_qubits)
            carries_clean = read_register(state, carry_qubits) == 0
            if not (s == a + b + cin and a_out == a
                    and int(state[0]) == cin and carries_clean):
                failures += 1
                print(f"FAIL: a={a} b={b} cin={cin} -> state {state}")
            checked += 1

assert checked == 512
assert failures == 0, f"{failures} input combinations failed"
print(f"{checked} input combinations checked, {failures} failures")
```

```
512 input combinations checked, 0 failures
```

The example, all six selected measured cases, and all 512 exhaustive input combinations pass. This verifies the 4-bit adder for every allowed unsigned input and both carry-in values under ideal simulation.

Sources

Supplied class notebook: QC_Ekert_FA1bit_and_2bit_examp.ipynb. The SUM, CARRY, and reverse-CARRY definitions are retained unchanged.
Vedral, Barenco, and Ekert, *Quantum Networks for Elementary Arithmetic Operations*, 1995 preprint (journal publication: Phys. Rev. A 54, 147, 1996): arXiv paper.
IBM Qiskit bit-ordering documentation; Qiskit Aer SamplerV2 reference.